

ADR-003: ingest `.zip` and `.7z`, and take a dependency to do it

Status: accepted, 2026-09-06 **Supersedes nothing. Constrained by ADR-001. Extended by ADR-007, which added `.rar` on 2026-11-01.**

Context

The roadmap promised ingest of "loose folders, `.sng`, and `.zip` / `.7z` of a loose folder", and the first two shipped in Phase 1. Community charts are distributed as archives far more often than as bare folders, so an operator's realistic first act is to drop a downloaded `.zip` into the library.

ADR-001 committed this project to a single static binary that cross-compiles to six targets with `CGO_ENABLED=0`. Until today its entire dependency set was one library, `golang.org/x/text`, for Unicode normalisation the standard library does not provide.

`.zip` costs nothing: `archive/zip` is in the standard library.

`.7z` has no standard-library reader and no plausible one. It is a container with a dozen possible compression methods (LZMA, LZMA2, BCJ filters, PPMd, Brotli, Deflate), and writing one is not a side quest inside a song server.

Decision

Ingest `.zip` with the standard library and `.7z` via `github.com/bodgit/sevenzip`.

What it actually costs, measured rather than estimated

`go list -m all` after adding it is alarming — it lists Google Cloud Storage, gRPC, OpenTelemetry and protobuf. **None of that is compiled in.** That output is the module *graph*: everything any dependency could theoretically reach. `spf13/afero`, which sevenzip uses, ships a GCS backend that nothing here imports.

The honest measures are the package list of the built command and the binary it produces:

	before	after
Third-party packages compiled into the server	3	71
`linux/arm64` binary	9,918,419 bytes	11,617,809 bytes
Anything from cloud.google.com / gRPC / OpenTelemetry	—	**not compiled in**

+1.62 MB, about 17%. The distroless image moves from roughly 8.6 MB to roughly 10.2 MB. All six release targets still cross-compile with `CGO_ENABLED=0`, verified by building each one.

Consequences

Good

- The shape an operator actually downloads works without unpacking it first.
- One code path. `archive/zip`'s `Reader` and sevenzip's `Reader` both already implement `fs.FS`, and the scanner has taken an `fs.FS` since Phase 1, so **no adapter and no duplicated scanning logic** was needed. A song in an archive is scanned by exactly the code that scans a loose folder.
- The container leaves no trace in the output. A song packs to the same `.sng` whether it is loose, zipped or 7z'd — asserted by `TestAllThreeContainersProduceTheSameArchive`. That is what lets an operator restructure a library without every client re-downloading every song.

Bad / accepted

- **The dependency posture changed, and this is the entry that admits it.** One library became eleven modules and 71 compiled packages. Every one is pure Go, so ADR-001's cross-compilation guarantee survives, but the supply-chain surface is not what it was and a future reader should not discover that by accident.
- 1.62 MB on a target whose whole appeal was an 8.6 MB image.
- `.7z` **support is read-only and untestable from Go.** The library cannot write archives, so the test fixture (`internal/scan/testdata/song.7z`) is committed rather than generated, and it was built with `py7zr` from exactly the bytes the loose-folder test uses. Regenerating it from anything else makes the test fail for reasons unrelated to 7z.

Rejected alternatives

- *`.zip` only, `.7z` refused with a message.* Zero cost and consistent with how console packages are handled, but it quietly redefines a promise rather than meeting it.
- *Shell out to `7z`.* Kills the single-static-binary property and adds a runtime requirement to every deployment target, including the Pi and the distroless image, which has no shell at all.
- *Vendor a minimal LZMA decoder.* Cheaper in modules, far more expensive in code we would then own and have to be correct about, on a format we do not control.

What is deliberately NOT ingested

`.con`, `_rb3con`, `.pkg` and `.xex` are **recognised and refused with a stated reason** rather than ignored. Decrypting Rock Band console packages is a permanent non-goal — upstream rejects it on sight and it carries real DMCA 1201 exposure — but an operator who drops a 2 GB `_rb3con` into the library and sees *nothing at all* concludes the server is broken. A refusal that says why costs one line.

Note `_rb3con` is matched as a **suffix**, not an extension: those files usually have no extension, so anything keying on `filepath.Ext` misses the most common shape.

An archive holding several songs is refused, not resolved

The promise is "a `.zip` of a loose folder", singular. An archive with two song folders raises `ErrTooManySongs`. Picking one would publish it and silently drop the rest, and the operator would have no way to discover that had happened.

One wrapper folder is expected and descended through — `Song.zip` containing `Song/` is how songs are actually distributed — up to a bounded depth, because an unbounded descent on a hostile archive is a denial of service.

Hostile archives, and the one defect that probing found

An operator's library is full of files downloaded from the internet, so the archive reader is attacker-facing. The behaviours below were **measured** — a throwaway test that printed what actually happens — before anything was asserted about them. Each is now pinned in `internal/scan/hostile_test.go`, because we depend on them and none of them are ours to control.

Path traversal never reaches the served archive. Entries named `../../etc/passwd`, `Song/../../escape.txt` and `/etc/shadow` are dropped by Go's zip filesystem, which refuses any name failing `fs.ValidPath`. We never extract to disk, so classic zip-slip does not apply, but the entry names we *emit* are attacker-influenced and reach every client. The test asserts both halves: the hostile names are absent from the produced `.sng`, and the legitimate files are still all present — a defence that also dropped the real content would be a different bug.

A corrupt archive is reported and the walk continues. One truncated `.zip` must not abandon a scan of ten thousand songs.

An archive that visibly holds a song is never silently ignored. This is the defect probing found. Some Windows tools write zip entries with **backslash** separators (`Song\song.ini`). Go's zip filesystem surfaces a directory for those but nothing readable underneath, so the chart is never found; because "no chart in a zip" is deliberately silent — libraries are full of archives that are not songs — a perfectly good song disappeared with no message at all. That is the same failure this ADR already rejected for `_rb3con`, arrived at from the other direction.

So `OpenContainer` now checks the **raw stored entry names** for a chart or `song.ini` before concluding the silence was justified, and reports `ErrUnreadableArchive` when one is visibly there. The first attempt at this walked the `fs.FS` view and was wrong for the obvious-in-hindsight reason: that view is exactly what is blind to these entries, so it inherited the blindness it existed to detect. The failing test is what said so.

We do not *read* backslash entries — that would mean second-guessing the archive's own name encoding, where a wrong guess re-introduces traversal through a separator Go does not treat as one. The operator is told the archive is unreadable and can re-zip it. Refusing loudly is the whole of the fix; reading it is a separate decision with a worse risk profile.

The counterpart is pinned too: an archive of genuinely unrelated files stays silent. If every holiday-photos zip became a problem entry, the real problems would be buried.
